

Almacenamiento de información

Introducción

XML es un estándar ampliamente utilizado para almacenar e intercambiar **información estructurada**.

Características principales:

- Separa **estructura** y **presentación** de los datos.
- Es texto plano, por lo que puede abrirse con cualquier editor.
- Es interoperable entre plataformas y tecnologías.
- Permite validación mediante **DTD** o **XML Schema (XSD)**.

Limitación importante:

- Para localizar información específica, normalmente es necesario procesar el documento completo o disponer de índices en el motor de almacenamiento.

Ámbitos de aplicación

Los documentos XML y sus requisitos de almacenamiento suelen clasificarse en **dos grupos**.

Sistemas centrados en datos

- Estructura bien definida y predecible.
- Datos actualizables y consultables con frecuencia.
- Casos típicos: facturas, pedidos, catálogos, inventario.

En este escenario, las bases de datos relacionales suelen ofrecer mejor rendimiento para operaciones *transaccionales*.

Sistemas centrados en documentos

- Tamaño y estructura más variables.
- Mayor heterogeneidad y presencia de campos opcionales.
- Casos típicos: artículos, publicaciones, normativa, documentación técnica.

En este escenario, los sistemas de gestión documental y las bases de datos XML nativas suelen encajar mejor.

XML y bases de datos

Podemos entender una base de datos XML como una **colección de documentos XML bien formados**. En muchos casos, los documentos siguen un mismo esquema, aunque no es obligatorio.

Ventajas de XML como formato de persistencia:

- Flexibilidad estructural.
- Buen soporte para interoperabilidad e intercambio entre sistemas.
- Modelo jerárquico natural para información *semiestructurada*.

No obstante, XML **no sustituye por sí mismo** a un **SGBD completo**. Sin un motor adecuado, faltan capacidades clave como:

- Índices eficientes.
- Control de concurrencia.
- Transacciones ACID.
- Seguridad avanzada.
- Integridad referencial y lógica compleja.

Sistemas de almacenamiento de la información

XML permite integrar **dos mundos tradicionalmente separados**:

- **Sistemas documentales (ficheros)**: estructura irregular, tipos simples, importancia del orden.
- **Sistemas estructurados (relacionales)**: estructura tabular, tipos complejos, menor importancia del orden.

Semejanzas entre XML + XSD y un SGBD:

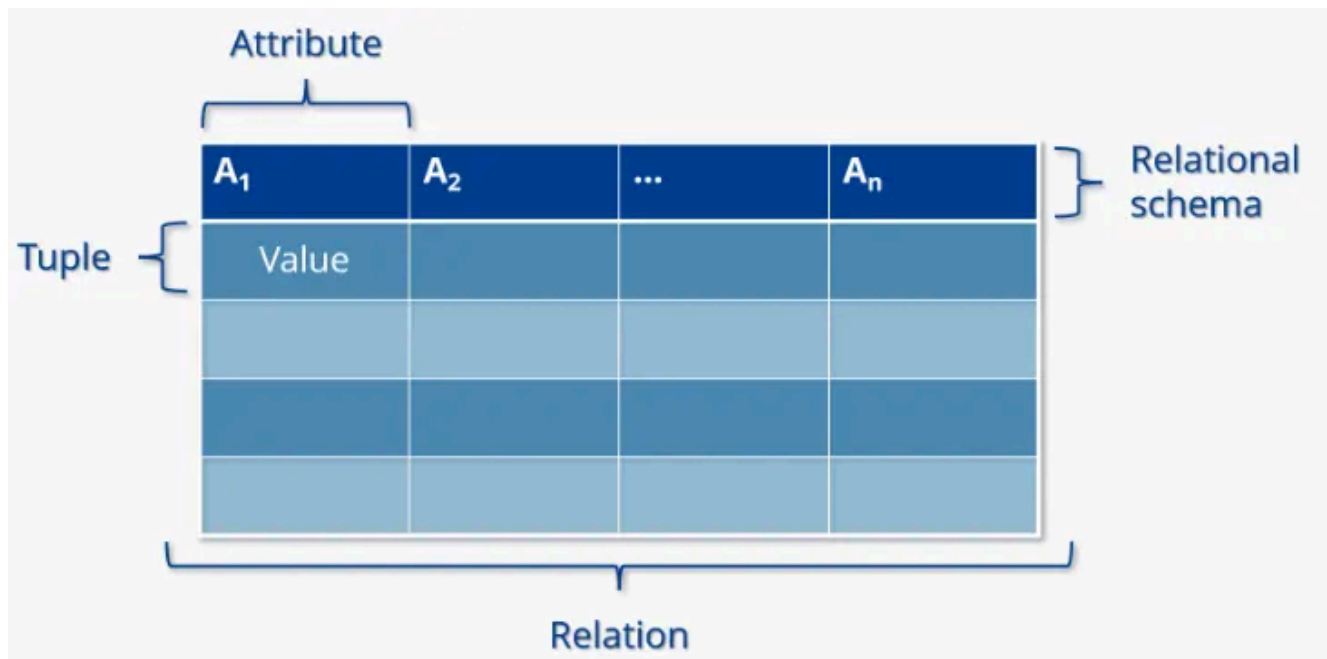
- Definición de esquemas.
- Lenguajes de consulta (XPath, XQuery).
- APIs de procesamiento (DOM, SAX, StAX).

Diferencias importantes:

- XML no aporta de forma nativa todas las garantías de un SGBD.
- El rendimiento en cargas transaccionales suele ser inferior sin estrategias de indexación y partición.

Bases de datos relacionales

Las bases de datos relacionales representan la información mediante tablas y se consultan habitualmente con **SQL**.



Características

Las bases de datos relacionales suponen una posibilidad para el almacenamiento de datos XML. Sin embargo, no están bien preparadas para almacenar estructuras de tipo jerárquico como son los documentos XML. Algunas de las causas son:

- Las bases de datos relacionales tienen una **estructura regular** frente al **carácter heterogéneo** de los documentos XML.
- Los documentos XML suelen contener muchos **niveles de anidamiento** mientras que los datos relacionales son **planos**.
- Los documentos XML tienen un **orden intrínseco** mientras que los **datos relacionales son no ordenados**.
- Los **datos relacionales son generalmente densos** (cada columna tiene un valor), mientras que los datos XML son **dispersos**, es decir, pueden representar la carencia de información mediante la ausencia del elemento.

Algunas de las razones para usar los tipos de bases de datos relacionales y los productos de bases de datos existentes para almacenar XML, aún cuando no sea de forma nativa son:

- Las bases de datos relacionales y orientadas a objetos son bien conocidas, mientras que las bases de datos XML nativas son nuevas.

- Como resultado de la familiaridad con las bases de datos relacionales y orientadas a objetos, los usuarios se inclinan a ellas especialmente por el rendimiento.

Middleware de transformación

El middleware permite transformar entre **estructura XML** y **modelo relacional**. El proceso de traducción puede ser descompuesto en los pasos mostrados a continuación.

Crear el XML Schema base

El XML Schema se crea siguiendo las siguientes directrices:

- Un **elemento** para cada **tabla**.
- Un **atributo** para cada **columna no clave**.
- Las columnas que **no permiten valores nulos** pueden ser marcadas como **requeridas**.
- Las columnas que **permiten valores nulos** pueden ser marcadas como **opcionales**.
- Las columnas pueden ser también **anidadas como elementos**, pero pueden surgir problemas cuando el mismo nombre de columna es usado en más de una tabla. Por ello, lo más simple es transformar las columnas como atributos XML, donde las colisiones de nombre en el esquema XML no son un problema.

Gestionar las claves primarias

El segundo paso es crear las claves primarias (*primary keys*) en el XML Schema. Una solución podría ser agregar un atributo para la columna clave primaria, con un ID agregado al nombre de la columna. Este atributo podría necesitar ser definido en el XML Schema como de tipo ID.

Pueden surgir **problemas de colisión** al crear claves primarias en el esquema XML, ya que a diferencia de las bases de datos relacionales, donde las claves primarias necesitan ser únicas sólo dentro de una tabla, un **atributo ID dentro de un documento XML debe ser único a través de todo el documento**. Para resolverlo, se puede agregar el nombre del elemento (nombre de la tabla), al valor de la clave primaria (valor del atributo). Esto asegura que el valor es único a través del documento XML.

Establecer las relaciones

Las relaciones entre tablas se pueden lograr mediante el **anidamiento de elementos bajo el elemento padre**. Un ID de XML Schema puede ser usado para apuntar a una estructura XML correspondiente conteniendo un IDREF. Pueden existir muchas variaciones de XML Schemas para representar la misma base de datos relacional.

Bases de datos orientadas a objetos

Las DBOO modelan directamente **objetos persistentes**.

Características:

- Cercanas a **lenguajes orientados a objetos**.
- Útiles cuando la aplicación trabaja con grafos de objetos complejos.
- Permiten mapear XML a objetos de forma natural.

Componentes habituales (estándares ODMG históricos):

- **Modelo de objetos**. Está concebido para proporcionar un modelo de objetos estándar para las bases de datos orientadas a objetos. Es el modelo en el que se basan los demás componentes.
- **Lenguajes de Especificación de Objetos (ODL)**. Lenguaje específico para la definición de objetos.
- **Lenguaje de Consulta de Objetos (OQL)**. Lenguaje específico para realizar consultas contra los objetos.
- **Lenguaje de Manipulación de Objetos (OML)**. Utilizado en lenguajes de programación. Extiende el lenguaje de programación para soportar objetos persistentes. Además, incluyen soporte para OQL, navegación y transacciones. El uso de este lenguaje es posible gracias a los bindings, los cuales se crean para los diferentes lenguajes de programación, como C++ o Java.

Bases de datos orientadas a objetos y XML

La transformación de un documento XML a objetos es inmediata. Una vez realizada la transformación, los objetos son gestionados directamente por el sistema gestor de la base de datos.

La información en los objetos creados se consulta acudiendo al lenguaje de consulta OQL.

Por otro lado, los mecanismos propios de un sistema gestor de base de datos (indexación, optimización, procesamiento de consultas, etc.) son los del propio sistema gestor de la base de datos y, por lo general, no son específicos para el modelo XML.

Bases de datos XML nativas

Son motores diseñados específicamente para **almacenar y consultar XML**.

Características habituales

Algunas de las características de las bases de datos XML nativas son:

- **Almacenamiento de documentos en colecciones:** las colecciones juegan en las bases de datos nativas el papel de las tablas en las base de datos relacionales.
- **Validación de los documentos.**
- **Consultas:** la mayoría de las bases de datos XML nativas soportan uno o más lenguajes de consulta. Uno de los más populares es **XQuery**.
- **Indexación XML:** se ha de permitir la creación de índices que aceleren las consultas realizadas.
- **Creación de identificadores únicos:** a cada documento XML se le asocia un identificador único.
- **Actualización y borrado de documentos XML.**

Tipos según almacenamiento #

Basado en texto #

Almacena el documento XML entero en forma de texto y proporciona alguna funcionalidad de base de datos para acceder a él.

Esto se puede conseguir de dos formas:

- Almacenar el documento en un almacén adecuado con índices, soporte para transacciones, etc.
- Almacenar el documento como un Blob en una base de datos relacional, mediante un fichero, y proporcionar algunos índices sobre el documento que aceleren el acceso a la información.

Basado en modelo #

Almacena un modelo binario del documento en un almacén existente o bien específico. Por ejemplo, almacenando el DOM.

Existen diferentes formas de almacenar la información de los documentos XML en binario:

- Traducir el DOM a tablas relacionales como elementos, atributos, entidades, etc.
- Traducir el DOM a objetos en una base de datos orientada a objetos.
- Utilizar un almacén creado especialmente para esta finalidad.